

mipi_2vc_stream_cmd

[TOC]

项目简介

mipi_2vc_stream_cmd文件夹下的sample提供了MIPI双虚拟通道出图时基础出图和通信的示例。

该示例展示了以下功能：

- 获取数据
- 显示出图
- 发送命令

该示例从上层（应用层）到下层（通信协议层）分别依赖了：

基本功能组件（commom文件夹）中的data ‘v4l2_camera ‘drm_display模块

特殊功能组件（components文件夹）中的cmd模块

接口适配层（interfaces文件夹）中的libircam ‘libircmd

驱动层（drivers文件夹）中的libiruart ‘libirv4l2。

程序编译方式

linux平台

mipi_2vc_stream_cmd文件夹下提供了CMakeList.txt文件，支持32位或64位sample程序的编译，相应的编译命令如下：

编译32位的sample

```
cmake -DBUILD32=ON
make
```

编译64位的sample

```
cmake -DBUILD32=OFF
make
```

注意：在sample编译之前，需要事先获取与sample相同位数（32位或64位）的libircam.so ‘libircmd.so ‘libiruart.so三个动态库，并将其放于drivers文件夹下，否则直接使用CMakeList.txt编译sample时会提示错误。

程序运行方式

./sample

如在RV1126平台上运行：./sample config/stream_RV1126.conf

修改内核驱动

本工程是基于RV1126的显示出图，需要修改内核以支持MIPI和UART。

本示例提供了编译好的zboot.img，位于RV1126_configure_files文件夹中。

更新内核

使用G1280s机芯出图时需要更新zboot.img.1280内核；使用MINI640机芯出图时需要更新zboot.img.640内核

以下以更新MINI640内核为例进行说明

```
adb push ./zboot.img.640 / #推送内核
adb shell #进入shell
dd if=zboot.img.640 of=/dev/mmcblk0p3 #更新内核，直接内核文件覆盖分区3即可
reboot #重启系统
```

MIPI出图节点

video5:MIPI0 virtual1

video6:MIPI0 virtual2

video7:MIPI0 virtual3

video8:MIPI0 virtual4

video11:MIPI1节点，需要先开启video11，才能使用下面的VC (另开一个shell，输入v4l2-ctl -d /dev/video11 --stream-mmap)

video12:MIPI1 virtual1

video13:MIPI1 virtual2

video14:MIPI1 virtual3

video15:MIPI1 virtual4

出图

出图功能使用的是V4L2框架，调用mipi驱动以获取数据

在sample.cpp的main函数中，以下代码设置MIPI对应device号，打开设备用于获取出图的handle。将会根据配置文件，打开图像流和温度流的设备

```
for (const auto &v4l2_config : product_config.camera.v4l2_streams)
{
    if (v4l2_config.type == "image")
    {
        printf("create image video handle\n");
        stream_frame_info.image_name = (char*)v4l2_config.device_name.data();
        ir_video_handle_create(&ir_image_video_handle);
        stream_frame_info.image_driver_handle = (Irv4l2VideoHandle_t*)irv4l2_handle_create(ir_image_video_handle);
        vc = (video_chan)(vc | IMAGE);
    }
    else if (v4l2_config.type == "temp")
    {
        printf("creat temp video handle\n");
        stream_frame_info.temp_name = (char*)v4l2_config.device_name.data();
        stream_frame_info.temp_driver_handle = (Irv4l2VideoHandle_t*)irv4l2_handle_create(ir_temp_video_handle);
        vc = (video_chan)(vc | TEMP);
    }
    else
    {
        printf("config err, not image video or temp video\n");
        return -1;
    }
}
```

在sample.cpp的main函数中，以下代码会寻找uart设备，打开并获取命令的handle。

```
printf("init control handle\n");
IrControlHandle_t* ir_control_handle = NULL;
ir_control_handle_create(&ir_control_handle);
IruartHandle_t* iruart_handle = NULL;
iruart_handle = iruart_handle_create(ir_control_handle);

FILE* fp;
char buffer[80] = { 0 };
char dev_node[20] = "/dev/";
fp = popen("ls /dev | grep \"ttyUSB*\" ", "r");
fgets(buffer, sizeof(buffer), fp);
printf("content of ls /dev | grep \"ttyUSB*\" is %s\n", buffer);
strcat(dev_node, buffer);
printf("dev_node is %s\n", dev_node);

int i;
for (i = sizeof(dev_node) - 1; i < sizeof(dev_node); i--)
{
    if (dev_node[i] == '\\n')
    {
        dev_node[i] = 0;
        break;
    }
}
ret = ir_control_handle->ir_control_open(iruart_handle, dev_node);
//set baudrate, params:115200 or 921600 or NULL(self-adaptation)
ret = ir_control_handle->ir_control_init(iruart_handle, NULL);
printf("open control node:%d\n", ret);
stream_frame_info.ircmd_handle = ircmd_create_handle(ir_control_handle);

//this function is used to judge whether the device data is ready
//if judge failed, any command is not allowed
ret = judge_device_status(stream_frame_info.ircmd_handle);
if (ret != 0)
{
    printf("judge_device_status failed\n");
    return -1;
}
```

```

}
ret = basic_video_stream_continue(stream_frame_info.ircmd_handle);
printf("basic_video_stream_continue:%d\n", ret);    //control mipi start

```

特别注意：

1. 对于uart串口波特率，未设置将默认使用115200波特率与机芯交互，如需设置其他波特率，见cmd.cpp文件内search_com函数内ir_control_init处调用
2. **judge_device_status**该函数用于判断固件flashdb数据是否写完。如果写完，则能读到完整的版本号信息，命令可以正常下发。如果没有写完，则可能读到错误的版本号信息，命令不能正常下发。该步骤不能省略，必须在判断成功后，后续步骤才能进行。

在config文件中，设置需要的出图模式，机芯会根据对应的模式设置相应分辨率，可选{ "yuyv_image", "nv12_image", "nv12_and_temp", "yuyv_and_temp", "uyvy_image" }

```

"camera":{
    ...
    "format":"nv12_and_temp",
    ...
}

```

本示例是按照MV12双出图情形来调用，双VC通道各自的分辨率如下：

- 图像数据：MipiVC0 width:640, height:387, fps:30
- 温度数据：MipiVC1 width:640, height:514, fps:30

如果需要调整分辨率，可以修改config目录下的stream.conf，配置参数说明见config目录下的stream.md：

- 选择对应的视频流video节点
- 在common/v4l2_camera.cpp的init_image_video_stream和init_temp_video_stream：使用config中的参数初始化数据通道

```

temp_dev_params.width = stream_frame_info->product_config.camera.v4l2_config.temp_stream.dev_width;
temp_dev_params.format = V4L2_PIX_FMT_YUYV;    //we transfer the data by composite format, only use V4L2_PIX_FMT_YUYV for 2
bytes transfer.
temp_dev_params.fps = stream_frame_info->product_config.camera.v4l2_config.temp_stream.fps;
temp_dev_params.height = stream_frame_info->product_config.camera.v4l2_config.temp_stream.dev_height;

```

图像显示

本示例使用drm显示BGR格式数据，所以在显示数据之前，需要对数据进行格式转换，使其成为BGR格式数据。

本实例支持两种数据格式转换方法

1. 使用RGA (Raster Graphic Acceleration Unit)库进行图像转换，RGA是一个独立的2D硬件加速器，可用于加速点/线绘制，执行图像缩放、旋转、bitBlt、alpha混合等常见的2D图形操作。该方法只是用于RV1126平台，且在该平台使用RGA库进行图像格式转换效率很高。（默认不使用该方案，若需使用需要重新编译可执行文件）

方式1. cmake设置编译选项

```

cmake -DWITH_RGA=ON
make

```

方式2. 运行cross_compilation_tool_chain.sh脚本 sh cross_compilation_tool_chain.sh

2. 使用common/transform.cpp的定义的图像格式转换接口进行格式转换，该方法在RV1126平台上转换效率没有RGA库高，但是在更多平台下进行图像数据转换。（默认方案）

发送命令

当运行sample之后，main函数会新建一个cmd_function线程，线程中包含了几个简单的调用命令示例，见cmd.cpp，根据输入的数字来发送命令。

```

//command thread function
void* cmd_function(void* threadarg)
{
    int isRUNNING = 1;
    int cmd;
    while (isRUNNING)
    {
        printf("please select cmd type\n");
        printf("1:get_device_info 2:basic_ffc_update 3:palette_operate 4:temp_measure 5:data_update\n");
        printf("6:set_image_scene_mode 7:image_effection\n");
        scanf("%d", &cmd);
        command_sel(cmd, (StreamFrameInfo_t*)threadarg);
    }
    printf("cmd thread exit!!\n");
    return NULL;
}

```